CONSULTING INTELLIGENCE PLATFORM

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF AI&DS

By

N GIREESH REDDY	**2510080021**
Y VINNEEL REDDY	**2510080044**
U SRIRAM CHOWDARY	**2510080038**
A SAI HEMESH REDDY	**2510080003**
A DHANUSH REDDY	**2510080018**

Under the Esteemed Guidance of

Dr. Laiphangbam Melinda
Assistant Professor
Department of AI&DS



K L (Deemed to be) University

DEPARTMENT OF AI&DS



Declaration

The Project Report entitled CONSULTING INTELLIGENCE PLATFORM is a record of Bonafide work of N GIREESH REDDY, Y VINNEEL REDDY, U SRIRAM CHOWDARY, A SAI HEMESH REDDY, A DHANUSH REDDY submitted in partial fulfillment for the award of B. Tech in AI&DS to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

K L (Deemed to be) University

DEPARTMENT OF AI&DS

CERTIFICATE

This is certify that the mini project based report entitled CONSULTING INTELLIGENCE PLATFORM is a bonafide work done and submitted by N GIREESH REDDY, Y VINNEEL REDDY, U SRIRAM CHOWDARY, A SAI HEMESH REDDY, A DHANUSH REDDY in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in AI&DS, K L (Deemed to be University), during the academic year 2025-2026.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

We would like to thank everyone who helped us complete this project. We are grateful to our guide [**Dr. Laiphangbam Melinda**] for constant support, guidance and encouragement. We thank our Course Coordinator and the Head of the Department for providing the facilities needed for the project. We also thank all teaching and non-teaching staff, and our parents, friends and classmates for their help and cooperation.

N GIREESH REDDY - 2510080021

Y VINNEEL REDDY - 2510080044

U SRIRAM CHOWDARY - 2510080038

A SAI HEMESH REDDY - 2510080003

A DHANUSH REDDY - 2510080018

ABSTRACT

Public services such as certificates, complaint handling and government notices are often delivered through separate and manual systems. This makes the process slow, unclear and inconvenient for citizens. This project, **Citizen Service Management with Unified Backend Architecture**, develops a digital platform that brings these services together under one scalable backend.

The system is organised into five modules: Citizen Management, Service Request Management, Complaint and Notification Management, REST API Development and Authentication, and Deployment, Testing and Documentation. Citizens can register, apply for services, file complaints and read notifications. Officers and administrators can review requests, resolve complaints and publish announcements.

The backend is built with Node.js, Express.js and MongoDB. Security is provided by JSON Web Token (JWT) authentication, password hashing and role-based access control. The APIs were tested with Postman and the system was deployed on the cloud. The result is a secure and easy-to-use platform that supports efficient public service delivery.

Keywords: Citizen services, unified backend, REST API, JWT authentication, MongoDB, Node.js.

INDEX

CHAPTER 1: INTRODUCTION **10**

1.1 Background of the Project 10

1.2 Problem Statement 10

1.3 Scope of the Project 11

1.4 Objectives of the Project 12

1.5 Importance of the Application 12

1.6 Target Users / Audience 13

1.7 Existing System versus Proposed System 13

1.8 Organization of the Report 13

CHAPTER 2: SYSTEM REQUIREMENTS **14**

2.1 Hardware Requirements 14

2.2 Software Requirements 14

2.3 Development Tools and Frameworks 14

2.4 Functional and Non-Functional Requirements 15

CHAPTER 3: TECHNOLOGY STACK **16**

3.0 Why These Technologies Were Chosen 16

3.1 Front-End 16

3.2 Back-End 16

3.3 Database 17

3.4 Version Control 17

3.5 APIs and External Services 18

CHAPTER 4: SYSTEM ARCHITECTURE **19**

4.1 High-Level Architecture Diagram 19

4.2 Description of Each Layer 19

4.3 Deployment Architecture 20

4.4 Security Architecture 20

CHAPTER 5: DESIGN **22**

5.1 ER Diagram 22

5.2 Database Schema 22

5.3 Use Cases 24
5.4 Request Flow 24

CHAPTER 6: IMPLEMENTATION 26

6.1 Module-wise Implementation 26
6.2 Front-End Logic 27
6.3 API Integration 27
6.4 Authentication and Authorization 28
6.5 Module Details: Inputs, Processing and Outputs 29
6.6 Sample API Request and Response 30
6.7 Validation and Error Handling 31

CHAPTER 7: FEATURES 32

7.1 List of Main Features 32
7.2 How Each Feature Works 32

CHAPTER 8: TESTING 34

8.1 Postman Testing 34
8.2 Integration Testing 34
8.3 Tools Used for Testing 35
8.4 Test Cases and Results 35
8.5 Security and Negative Testing 35

CHAPTER 9: DEPLOYMENT 37

9.1 Steps to Deploy 37
9.2 Environment Configuration 37
9.3 Hosting of Front-End and Back-End 37

CHAPTER 10: CHALLENGES AND LIMITATIONS 38

10.1 Issues Faced During Development 38
10.2 Solutions Applied 38
10.3 Current Limitations 38

CHAPTER 11: FUTURE ENHANCEMENTS 39

11.1 Planned Features 39
11.2 Possible Integrations and Optimizations 39

CHAPTER 12: CONCLUSION **40**

12.1 Summary of the Project 40

12.2 What Was Achieved 40

12.3 Skills Learned During Development 40

12.4 Project Work Plan 41

CHAPTER 13: REFERENCES **42**

CHAPTER 14: APPENDICES **43**

A. Sample Code Snippets 43

B. Installation and Setup Instructions 44

C. User Manual 44

D. Simple Glossary 45

E. Screenshots 45

F. Geo-tagged Photos and Review Forms 48

LIST OF FIGURES

Figure 4.1: High-level architecture of the system

Figure 4.2: Deployment flow from developer to live system

Figure 5.1: ER diagram of the system

Figure 5.2: Journey of one API request

Figure 6.1: Life cycle of a service request

LIST OF ABBREVIATIONS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
ER	Entity Relationship
HTTP / HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JWT	JSON Web Token
REST	Representational State Transfer
UI	User Interface

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Every citizen depends on public services in daily life. People apply for birth and income certificates, pay municipal bills, report broken roads or water leaks, and look for government announcements. In most places these tasks still depend on paper forms, long queues and repeated visits to government offices. A citizen often does not know whether an application has been received, who is handling it, or how long it will take.

Many departments have started to use computers, but each department usually keeps its own separate system. One system handles certificates, another handles complaints and a third sends notices. The data is scattered and the systems do not talk to each other. This makes it hard for officers to track work and hard for citizens to get information.

> In simple words: a backend is the hidden part of a website or app that stores data and applies the rules. The part you see on screen is called the frontend. A unified backend means one single backend that serves every service, instead of many separate ones.

This project builds one digital platform, called the Citizen Service Management System, with a single unified backend. Through this backend, citizens can register, apply for services and certificates, file complaints, and receive government notifications. Officers and administrators use the same backend to review requests, update status and send announcements.

The backend follows a scalable design. Scalable means the system keeps working well when the number of users grows from a few hundred to many thousands. This is achieved by keeping the code in separate modules, using a REST API for communication, and storing data in a database that can grow.

1.2 Problem Statement

Citizens face the following problems when they use public services today:

- Services are offered through many separate systems, so there is no single place to apply and track.

- Citizens must visit offices in person, which wastes time and money.
- There is no clear tracking. A citizen cannot easily see the current status of an application or complaint.
- Complaints are often lost or delayed because nobody is clearly responsible for them.
- Important notices reach people late, or never reach them.
- Paper records are hard to search and can be lost, damaged or misused.

Because of these problems, there is a need for one secure and easy-to-use platform that brings citizen records, service requests, complaints and notifications together under one backend system. This project is developed to solve that need.

1.3 Scope of the Project

The scope tells us what the project covers and what it does not.

Included in the project:

- Citizen registration, login and profile management.
- Online applications for services such as certificates, with document details and status tracking.
- Complaint filing, assignment to an officer and resolution tracking.
- Government notifications and announcements sent to citizens.
- A secure REST API with token-based authentication and role-based access (citizen, officer, admin).
- Testing of the API, deployment on the internet and project documentation.

Not included in this version:

- Real online payment gateway integration.
- Integration with real government databases such as Aadhaar.
- A mobile application (the API is ready to support one in future).

1.4 Objectives of the Project

- To build one unified backend that manages citizens, service requests, complaints and notifications.
- To let citizens apply for services and file complaints online from anywhere.
- To let citizens track the status of every request in real time.
- To help officers and administrators handle requests in an organised way.
- To protect user data using secure login, password encryption and role-based access.
- To design clean REST APIs that any frontend or mobile app can use later.
- To test, deploy and document the system so that others can understand and reuse it.

1.5 Importance of the Application

This application is important for the following reasons:

- **Saves time and cost:** citizens do not have to travel to offices for simple tasks.
- **Transparency:** every request has a visible status and history, which reduces delay and unfair practices.
- **Accountability:** each complaint and request is assigned to a specific officer.
- **Faster communication:** notifications reach citizens at once instead of through notice boards.
- **Better record keeping:** digital records are easy to search, back up and protect.
- **Supports Digital India:** the project follows the national goal of delivering public services electronically.

1.6 Target Users / Audience

Table 1.1: Types of users of the system

User	Who they are	What they do in the system
Citizen	Any registered resident	Registers, applies for services, files complaints, reads notifications
Officer	Government staff member	Reviews requests, resolves complaints, updates status
Administrator	Department head or system manager	Manages users, assigns work, sends notifications, views reports

1.7 Existing System versus Proposed System

Table 1.2: Comparison of the existing and proposed systems

Point	Existing system	Proposed system
Applying for a service	Visit the office and fill a paper form	Apply online from any place at any time
Tracking	Ask at the counter or call repeatedly	See the live status after login
Complaints	Written application, often lost or delayed	Online complaint with id, category and time stamp
Responsibility	Not clear who is handling the case	Every case is assigned to a named officer
Notices	Notice boards and newspapers	Instant notifications inside the platform
Records	Paper files, hard to search	Digital records in a database, easy to search and back up
Security	Files can be lost or misused	Login, encryption and role-based access
Systems	Separate system for each task	One unified backend for all tasks

1.8 Organization of the Report

The rest of this report is arranged as follows. Chapter 2 lists the system requirements. Chapter 3 explains the technology stack. Chapter 4 describes the system architecture. Chapter 5 presents the design, including the database. Chapter 6 explains how the modules were implemented. Chapter 7 describes the features. Chapter 8 covers testing and Chapter 9 covers deployment. Chapter 10 discusses challenges and limitations, Chapter 11 suggests future enhancements and Chapter 12 concludes the report. References and appendices follow.

CHAPTER 2: SYSTEM REQUIREMENTS

This chapter lists the hardware and software needed to build and run the project. These are the minimum needs for a normal student laptop.

2.1 Hardware Requirements

Table 2.1: Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 or higher
RAM	4 GB	8 GB or more
Storage	10 GB free space	256 GB SSD
Internet	Required for deployment and testing	Stable broadband
Display	1366 x 768	1920 x 1080

2.2 Software Requirements

Table 2.2: Software requirements

Software	Purpose
Windows 10/11, Linux or macOS	Operating system for development
Node.js (version 18 or above)	Runs the backend code
MongoDB / MongoDB Atlas	Database to store all data
Postman	Tool used to test the APIs
Git and GitHub	Version control and code storage
Google Chrome or Edge	Web browser for testing the frontend

2.3 Development Tools and Frameworks

Table 2.3: Tools used during development

Tool	Use
Visual Studio Code	Code editor used to write the project
Express.js	Framework used to build the REST API
Mongoose	Connects Node.js code to MongoDB and defines data models
Postman	Sends test requests to the API and checks the responses
GitHub	Stores the code online and tracks changes
Render	Hosts the backend on the internet

2.4 Functional and Non-Functional Requirements

Functional requirements describe what the system must do:

- The system shall allow a user to register and log in securely.
- The system shall allow a citizen to apply for a service and view its status.
- The system shall allow a citizen to file a complaint under a category.
- The system shall allow officers to update the status of requests and complaints.
- The system shall allow admins to manage users and publish notifications.
- The system shall restrict each action to the correct user role.

Non-functional requirements describe how well the system must work:

Table 2.4: Non-functional requirements

Requirement	Meaning in this project
Security	Passwords are hashed, tokens are signed and every protected route checks the role
Performance	Normal requests should be answered within about two seconds
Scalability	New modules and more users can be added without rewriting the system
Reliability	Errors are handled centrally so that the server does not crash on bad input
Usability	Pages and messages should be simple enough for first-time users
Maintainability	Code is split into models, controllers, routes and middleware

CHAPTER 3: TECHNOLOGY STACK

A technology stack is the set of tools and languages used to build a project. We chose tools that are free, popular and easy for students to learn.

3.0 Why These Technologies Were Chosen

- **Free and open source:** every tool in the stack can be used without paying.
- **One language:** JavaScript is used on both the front-end and the back-end, so the learning load is small.
- **Large community:** a student who gets stuck can easily find tutorials and answers.
- **Industry use:** the same stack is used in many real companies, which makes the skills useful for jobs.
- **Easy cloud deployment:** Render and MongoDB Atlas both offer free plans suitable for student projects.

3.1 Front-End

The front-end is the screen that the citizen sees. It is built with HTML for structure, CSS for design and JavaScript (React.js) for interaction. It contains pages for registration, login, service application, complaint form, status tracking and notifications. The front-end never stores secrets. It only sends requests to the backend API and shows the answers.

3.2 Back-End

The back-end is built with Node.js and Express.js.

- **Node.js** lets us run JavaScript on the server. It is fast and handles many users at the same time.
- **Express.js** is a small framework on top of Node.js that makes it easy to create API routes such as /api/complaints.
- Because both front-end and back-end use JavaScript, students need to learn only one language.

All five modules of the project live inside this one backend. Each module has its own routes, controllers and models, but they share the same database connection, security rules and error handling. This is what makes the architecture unified.

3.3 Database

We use MongoDB, a NoSQL database. Instead of tables with rows, it stores data as documents that look like JSON objects. Each document is kept in a collection, for example the citizens collection or the complaints collection.

> In simple words: think of MongoDB as a set of folders (collections). Each folder holds many digital forms (documents). A form can have different fields, so it is flexible when requirements change.

MongoDB was chosen because it is easy to scale, works well with Node.js through Mongoose, and offers a free cloud service called MongoDB Atlas for deployment.

3.4 Version Control

Git is used to record every change made to the code. GitHub stores the project online so that team members can work together. Each module was developed in its own branch and then merged into the main branch after testing. If a mistake happens, we can return to an older working version.

3.5 APIs and External Services

Table 3.1: Libraries and services used

Name	Purpose
REST API (Express routes)	Lets the front-end and backend exchange data using standard HTTP methods
JSON Web Token (JWT)	Creates a secure login token after a user logs in
bcrypt	Converts passwords into unreadable hashes before saving
dotenv	Keeps secret values such as database links out of the code
cors	Allows the front-end website to call the backend safely
express-validator	Checks that user input is correct and safe
Nodemailer (optional)	Sends email notifications
MongoDB Atlas	Cloud database service
Render	Cloud hosting for the backend

In simple words: an API is a waiter between two programs. The front-end places an order (request), the waiter carries it to the kitchen (backend), and brings back the food (response). REST is a common set of rules for how this ordering is done, using the methods GET (read), POST (create), PUT (update) and DELETE (remove).

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a layered architecture. Each layer has one clear job and talks only to the layer next to it. All users, whether citizen, officer or admin, reach the same single backend.

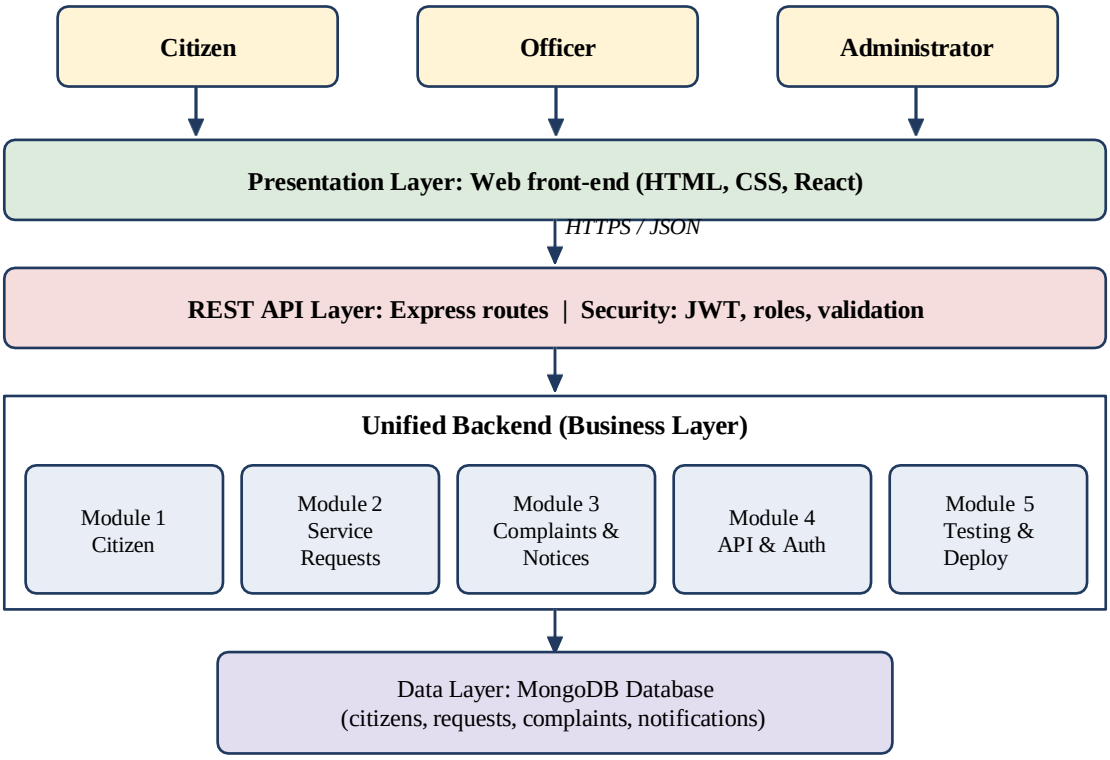


Figure 4.1: High-level architecture of the Citizen Service Management System

4.2 Description of Each Layer

Presentation layer (front-end). This is the web interface shown in the browser. It collects input from the user, sends it to the API and displays the result. It contains no business rules.

API layer (routes). Every request first reaches a route such as POST /api/complaints. The route decides which function should handle it. Before the function runs, security checks are applied.

Security layer (middleware). Middleware is a small function that runs between the request and the real work. Our middleware checks the JWT token (authentication), checks the role

of the user (authorization) and validates the input data.

Business layer (controllers and services). This layer holds the real logic of the five modules: registering a citizen, creating a service request, assigning a complaint, sending a notification and so on.

Data layer (database). MongoDB stores all permanent information in collections: citizens, service requests, complaints and notifications.

4.3 Deployment Architecture

The project is deployed on the cloud so that anyone can reach it through a web address.

- The backend code is stored on GitHub.
- Render is connected to the GitHub repository. Whenever new code is pushed to the main branch, Render builds and starts the application automatically. This is a simple CI/CD pipeline (continuous integration and continuous deployment).
- The database runs on MongoDB Atlas, which is connected to the backend through a secret connection string stored as an environment variable.
- The front-end is hosted separately on a static hosting service and calls the backend API over HTTPS.

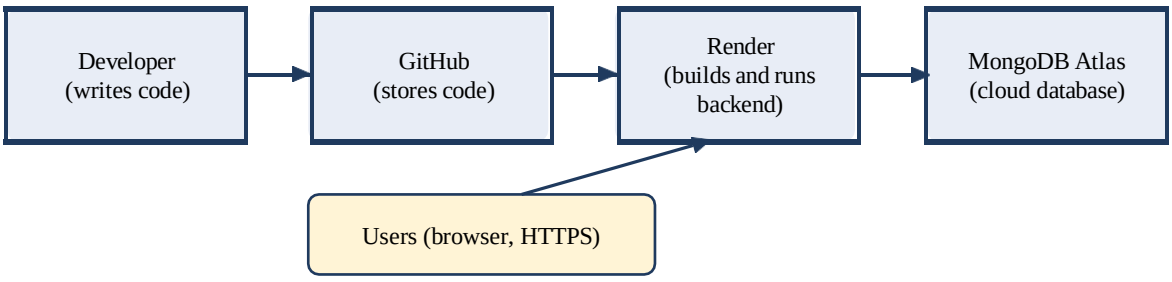


Figure 4.2: Deployment flow from developer to live system

4.4 Security Architecture

Security is built into every layer instead of being added at the end. The table shows which threat is handled where.

Table 4.1: Security measures

Threat	Protection used	Where applied
Stolen or weak passwords	bcrypt hashing; passwords never stored as plain text	Citizen module
Pretending to be another user	JWT token checked on every protected request	Auth middleware
Using actions not meant for the role	Role-based authorization	Role middleware
Bad or harmful input	Input validation with express-validator	Validation layer
Data stolen in transit	HTTPS encryption	Deployment
Secrets leaking in code	Environment variables, .env not uploaded to GitHub	Configuration
Calls from unknown websites	CORS allows only the trusted front-end	Server settings

CHAPTER 5: DESIGN

Design means planning how the system will look and work before writing code. This chapter shows the main database collections, how they are related, and how a request moves through the system.

5.1 ER Diagram

An ER (Entity Relationship) diagram shows the main data items (entities) and how they are connected. One citizen can make many service requests, file many complaints and receive many notifications. An officer can handle many requests and complaints.

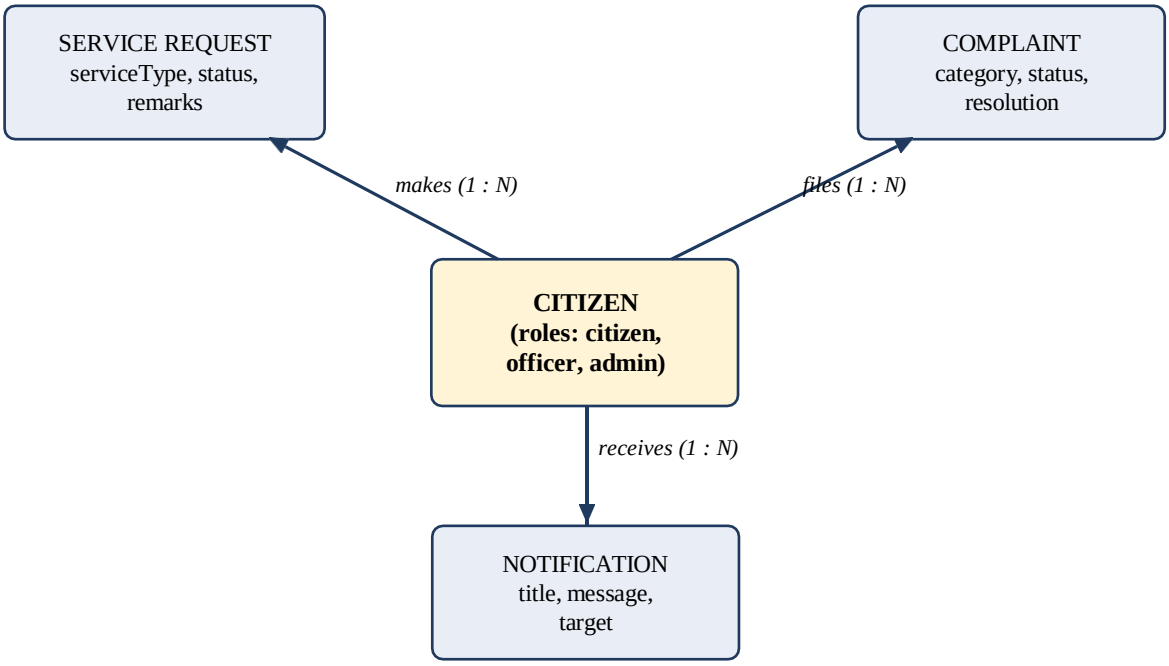


Figure 5.1: ER diagram of the system

5.2 Database Schema

The following tables list the fields stored in each collection.

Table 5.1: Citizen collection

Field	Type	Description
_id	ObjectId	Unique id created automatically
name	String	Full name of the user
email	String	Unique email used to log in
password	String	Hashed password (never plain text)
phone	String	Mobile number
address	String	Residential address
role	String	citizen, officer or admin
createdAt	Date	Registration date

Table 5.2: ServiceRequest collection

Field	Type	Description
_id	ObjectId	Unique request id
citizen	ObjectId	Reference to the citizen who applied
serviceType	String	Birth certificate, income certificate, etc.
details	String	Extra information given by the citizen
status	String	Submitted, In Review, Approved, Rejected
assignedTo	ObjectId	Officer handling the request
remarks	String	Officer's comments
createdAt / updatedAt	Date	Time stamps

Table 5.3: Complaint collection

Field	Type	Description
_id	ObjectId	Unique complaint id
citizen	ObjectId	Citizen who filed it
category	String	Water, Roads, Electricity, Sanitation, Other
description	String	Details of the problem
status	String	Open, In Progress, Resolved
assignedTo	ObjectId	Officer responsible
resolution	String	Action taken
createdAt / updatedAt	Date	Time stamps

Table 5.4: Notification collection

Field	Type	Description
_id	ObjectId	Unique notification id
title	String	Short heading
message	String	Full announcement text
target	String	all, or a specific citizen
createdBy	ObjectId	Admin or officer who sent it
readBy	Array	List of citizens who have read it
createdAt	Date	Time of sending

5.3 Use Cases

A use case describes one goal that a user wants to achieve with the system.

Table 5.5: Main use cases

Use case	Actor	Short description
Register / Login	Citizen, Officer, Admin	User creates an account or signs in and receives a token
Apply for service	Citizen	Citizen selects a service, fills details and submits
Track request	Citizen	Citizen views the current status and remarks
Review request	Officer	Officer opens a request and approves or rejects it
File complaint	Citizen	Citizen reports a public problem under a category
Assign complaint	Admin	Admin gives the complaint to a responsible officer
Resolve complaint	Officer	Officer records the action taken and closes the complaint
Publish notification	Admin, Officer	Announcement is created and shown to citizens
Manage users	Admin	Admin views, blocks or creates users

5.4 Request Flow

Every request in the system follows the same path. This makes the code easy to understand and to test.

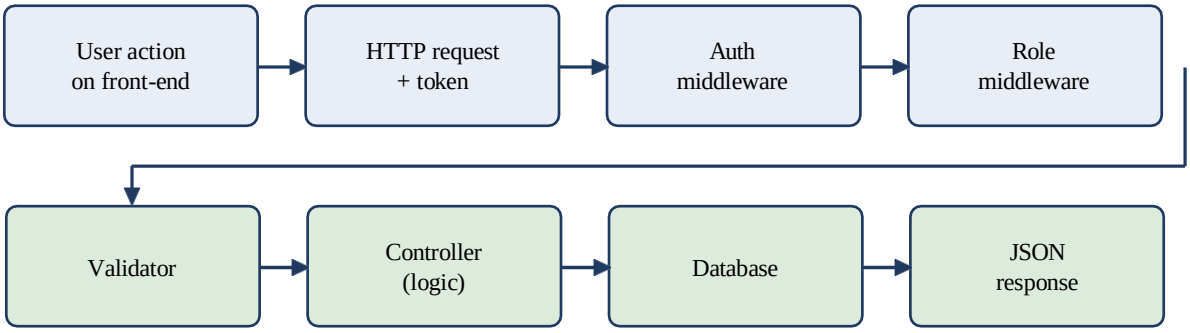


Figure 5.2: Journey of one API request

1. The user performs an action on the front-end, for example clicks 'Submit Complaint'.
2. The front-end sends an HTTP request to the API with the login token.
3. The authentication middleware checks that the token is valid.
4. The authorization middleware checks that the user's role is allowed to do this action.
5. The validator checks that the data is complete and correct.
6. The controller runs the business logic and talks to the database.
7. The API sends back a JSON response and the front-end shows the result.

CHAPTER 6: IMPLEMENTATION

This chapter explains how each module was built. The project is organised into folders so that each part has a clear place.

```
citizen-service-backend/
  config/         db.js (database connection)
  models/         Citizen.js, ServiceRequest.js, Complaint.js, Notification.js
  controllers/    authController.js, serviceController.js,
                  complaintController.js, notificationController.js
  routes/         authRoutes.js, serviceRoutes.js,
                  complaintRoutes.js, notificationRoutes.js
  middleware/     auth.js, role.js, errorHandler.js
  .env            secret settings (not uploaded to GitHub)
  server.js       starting point of the application
```

6.1 Module-wise Implementation

Module 1: Citizen Management. This module handles the people who use the system. A new user registers with name, email, phone, address and password. The password is hashed with bcrypt before saving. On login the system checks the email and password and returns a JWT token. Citizens can view and update their profile. Admins can view, block or delete users and can create officer accounts.

Module 2: Service Request Management. A citizen selects a service, such as an income certificate, fills in the details and submits. The system saves the request with the status 'Submitted' and links it to the citizen. An officer opens the request, reviews it and changes the status to In Review, Approved or Rejected with remarks. Each change updates the time stamp so that the full history can be seen by the citizen.

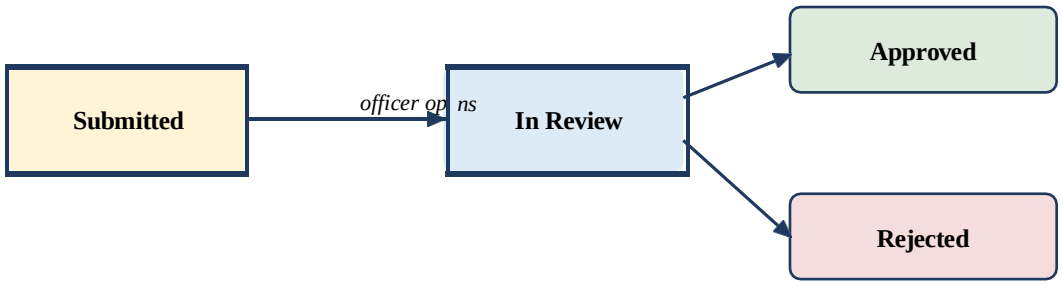


Figure 6.1: Life cycle of a service request

Module 3: Complaint and Notification Management. A citizen files a complaint under a category (for example Water or Roads). An admin assigns it to an officer. The officer

updates the status from Open to In Progress and finally to Resolved with a note about the action taken. Separately, admins and officers can publish notifications such as holiday notices, new scheme announcements or maintenance alerts. Citizens read them in their notification list, and the system records who has read what.

Module 4: REST API Development and Authentication. All the above features are exposed through REST API endpoints. Access is protected by JWT authentication and role-based authorization. This module also contains the input validation and central error handling.

Module 5: Deployment, Testing and Documentation. The API was tested in Postman, deployed to Render with MongoDB Atlas, and documented in this report and in the project README file.

6.2 Front-End Logic

The front-end is kept simple. After login, the JWT token is saved in the browser and sent with every API call in the Authorization header. The pages show different menus depending on the user's role. Forms check the input before sending it, for example whether the email format is correct. Lists such as 'My Requests' and 'My Complaints' are loaded from the API and displayed in tables with status labels. If the API returns an error, a clear message is shown to the user.

6.3 API Integration

The API endpoints are the heart of the backend. They follow REST rules: nouns for resources, HTTP methods for actions.

Table 6.1: Main API endpoints

Method	Endpoint	Purpose	Who can use
POST	/api/auth/register	Create a new account	Everyone
POST	/api/auth/login	Log in and get token	Everyone
GET	/api/citizens/me	View own profile	Logged-in user
PUT	/api/citizens/me	Update own profile	Logged-in user
GET	/api/citizens	List all citizens	Admin
POST	/api/services	Apply for a service	Citizen
GET	/api/services	List own requests (all for officer)	Logged-in user
PUT	/api/services/:id	Update request status	Officer, Admin
POST	/api/complaints	File a complaint	Citizen
GET	/api/complaints	List complaints	Logged-in user
PUT	/api/complaints/:id	Assign or resolve complaint	Officer, Admin
POST	/api/notifications	Send a notification	Admin, Officer
GET	/api/notifications	Read notifications	Logged-in user

Each response is sent in JSON format with a proper HTTP status code. For example 200 means success, 201 means created, 400 means bad input, 401 means not logged in, 403 means not allowed and 404 means not found.

6.4 Authentication and Authorization

Authentication means proving who you are. **Authorization** means checking what you are allowed to do. Both are essential for a system that holds citizen data.

1. A user logs in with email and password.
2. The server compares the password with the stored bcrypt hash.
3. If it matches, the server creates a JWT token that contains the user id and role and expires after a fixed time.
4. The user sends this token with every later request.
5. The auth middleware verifies the token. If it is missing or invalid the server returns 401.

6. The role middleware checks the role. For example only an officer or admin can change a request status. Otherwise the server returns 403.

Other security measures include hiding secret keys in the .env file, validating all input to block bad data, and allowing only trusted websites through CORS.

6.5 Module Details: Inputs, Processing and Outputs

The tables below summarise what each module receives, what it does with it and what it returns. This helps readers understand the logic without reading the code.

Table 6.2: Citizen Management module

Item	Details
Input	Name, email, phone, address, password
Validation	Email format is correct, password has at least 6 characters, email is not already used
Processing	Hash the password, save the citizen, create a token at login
Output	Success message and JWT token; profile data when requested
Errors	400 for bad input, 401 for wrong password

Table 6.3: Service Request Management module

Item	Details
Input	Service type and details from the citizen; new status and remarks from the officer
Validation	Service type is from the allowed list; status is one of the allowed values
Processing	Save request as Submitted; later update status, remarks and time stamp
Output	Request record with id, status and history
Errors	403 if a citizen tries to change the status; 404 if the request does not exist

Table 6.4: Complaint and Notification Management module

Item	Details
Input	Category and description from the citizen; officer id from the admin; resolution note from the officer; title and message for notifications
Validation	Category is from the allowed list; description is not empty
Processing	Save complaint as Open; assign officer; update status; save notification; record read status
Output	Complaint record with status; list of notifications
Errors	403 if the role is not allowed; 400 if required text is missing

6.6 Sample API Request and Response

The following example shows a citizen filing a complaint. The request is sent to the API and the server replies with the saved record.

Request: POST /api/complaints (header: Authorization: Bearer <token>)

```
{
  "category": "Water",
  "description": "No water supply in Ward 5 for three days"
}
```

Response: status 201 Created

```
{
  "_id": "6650f1c2a9b3e4d1c8f00a12",
  "citizen": "664fe0a1b7c2d3e4f5a60b78",
  "category": "Water",
  "description": "No water supply in Ward 5 for three days",
  "status": "Open",
  "createdAt":  "2025-05-24T10:15:30.000Z"
}
```

Error example: if the token is missing, the server replies with status 401 and this message:

```
{ "message": "Login required" }
```

Sample login request: POST /api/auth/login

```
{ "email": "ravi@example.com", "password": "secret123" }
```

Login response: status 200 OK

```
{  "token":  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  "role":  "citizen"  }
```

6.7 Validation and Error Handling

Good systems expect mistakes. In this project every request is checked before it reaches the database.

- **Validation:** required fields, email format, allowed values for status and category are checked using express-validator. If a rule fails, the server returns status 400 with a clear message.
- **Central error handler:** one function at the end of the application catches every unexpected error, writes it to the log and sends a safe message. Users never see internal details.
- **Not-found handling:** a request to an unknown route returns status 404.
- **Consistent format:** every response contains a message, so the front-end can show it without extra code.

CHAPTER 7: FEATURES

7.1 List of Main Features

- Secure registration and login for citizens, officers and admins.
- Role-based dashboards and access control.
- Online application for public services and certificates.
- Real-time status tracking of every request and complaint.
- Complaint filing with category and automatic time stamping.
- Assignment of complaints and requests to responsible officers.
- Government notifications with read tracking.
- Central error handling and input validation.
- Clean REST API that can also power a mobile app.

7.2 How Each Feature Works

Table 7.1: Working of the main features

Feature	What the user sees	What happens behind the scenes
Registration	Fills a form and gets a success message	Data is validated, password hashed, new citizen document saved
Login	Enters email and password and reaches dashboard	Password checked, JWT token created and returned
Apply for service	Selects service, adds details, submits	Request saved with status Submitted and linked to the citizen
Track status	Sees 'In Review' or 'Approved' in My Requests	API reads the request and returns the current status and remarks
File complaint	Chooses category, writes problem, submits	Complaint saved as Open with time stamp
Resolve complaint	Officer enters action taken and marks Resolved	Status and resolution updated, citizen sees the change
Notifications	Bell icon shows new announcements	Notification saved; reading it adds the citizen to readBy
Admin management	Admin manages users and assigns work	Admin-only routes protected by role middleware

Example walk-through. Ravi needs an income certificate. He registers and logs in, selects 'Income Certificate', adds his details and submits. The system gives him a request id and the status 'Submitted'. An officer opens the request, checks it and marks it 'Approved' with a remark. The next time Ravi opens his dashboard he sees 'Approved'. The whole process happened without a single office visit.

CHAPTER 8: TESTING

Testing means checking that the system works correctly and safely. We tested every API endpoint before deployment.

8.1 Postman Testing

Postman is a tool that sends requests to an API and shows the response. For each endpoint we created a request, sent sample data and checked the status code and the returned data. The requests were saved in a Postman collection so they can be repeated after every code change.

Typical steps followed in Postman:

1. Register a new user with POST /api/auth/register and check for status 201.
2. Log in with POST /api/auth/login and copy the token from the response.
3. Add the token in the Authorization header as 'Bearer <token>'.
4. Call the protected endpoints and check that correct data is returned.
5. Call them again without a token or with the wrong role and check that 401 or 403 is returned.

8.2 Integration Testing

Integration testing checks whether the modules work together. We tested complete user journeys instead of single functions.

- Register, log in, apply for a service, officer approves it, citizen sees the Approved status.
- Citizen files a complaint, admin assigns it, officer resolves it, citizen sees the resolution.
- Admin sends a notification, citizen reads it, read status is stored.

8.3 Tools Used for Testing

Table 8.1: Testing tools

Tool	Use
Postman	Manual API testing and saved test collections
Browser developer tools	Checking front-end requests and errors
MongoDB Atlas / Compass	Checking that data is saved correctly
Console logs	Finding errors while the server runs

8.4 Test Cases and Results

Table 8.2: Sample test cases

No.	Test case	Expected result	Result
1	Register with valid details	Account created (201)	Pass
2	Register with an existing email	Error message (400)	Pass
3	Login with correct password	Token returned (200)	Pass
4	Login with wrong password	Access denied (401)	Pass
5	Open profile without token	Access denied (401)	Pass
6	Citizen applies for a service	Request saved as Submitted (201)	Pass
7	Citizen tries to approve a request	Forbidden (403)	Pass
8	Officer approves a request	Status updated (200)	Pass
9	Citizen files a complaint	Complaint saved as Open (201)	Pass
10	Admin sends a notification	Notification created (201)	Pass
11	Citizen reads notifications	List returned (200)	Pass
12	Request with missing fields	Validation error (400)	Pass

All test cases passed. The tests confirmed that the system accepts correct input, rejects wrong input and protects restricted actions.

8.5 Security and Negative Testing

Negative testing means trying wrong or harmful actions on purpose to confirm that the system refuses them.

Table 8.3: Security test cases

No.	Test case	Expected result	Result
13	Use an expired token	Access denied (401)	Pass
14	Change one character of a token	Invalid token (401)	Pass
15	Citizen calls the admin-only user list	Forbidden (403)	Pass
16	Citizen opens another citizen's request	Not shown or forbidden	Pass
17	Send an invalid status value	Validation error (400)	Pass
18	Register with a very short password	Validation error (400)	Pass
19	Call an unknown route	Not found (404)	Pass
20	Check the database for a saved password	Only a hash is stored	Pass

CHAPTER 9: DEPLOYMENT

Deployment means putting the finished project on the internet so that other people can use it.

9.1 Steps to Deploy

1. Push the final project code to a GitHub repository.
2. Create a free cluster on MongoDB Atlas and copy its connection string.
3. Create a new Web Service on Render and connect it to the GitHub repository.
4. Set the build command to 'npm install' and the start command to 'node server.js'.
5. Add the environment variables in the Render dashboard (see Section 9.2).
6. Click Deploy. Render builds the project and gives a public URL.
7. Test the live URL with Postman to confirm that it works.

9.2 Environment Configuration

Secret and changing values are never written inside the code. They are kept as environment variables.

Table 9.1: Environment variables

Variable	Meaning
PORT	Port number on which the server runs
MONGO_URI	Connection string of the MongoDB database
JWT_SECRET	Secret key used to sign login tokens
JWT_EXPIRE	How long a token stays valid, for example 1d
CLIENT_URL	Address of the front-end allowed by CORS

9.3 Hosting of Front-End and Back-End

The back-end is hosted on Render and the database on MongoDB Atlas. The front-end is hosted on a static hosting service. The front-end calls the back-end URL using HTTPS, so all data travels in encrypted form. Because Render redeploys automatically whenever the main branch changes, updates reach the live system quickly.

CHAPTER 10: CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

- Understanding how JWT tokens and middleware work together was difficult at first.
- Different roles needed different permissions, and mistakes caused users to see data not meant for them.
- CORS errors blocked the front-end from calling the deployed API.
- Database connection failed after deployment because the cloud database did not allow the server's address.
- Keeping the API responses consistent across all five modules took extra effort.

10.2 Solutions Applied

Table 10.1: Problems and solutions

Problem	Solution
JWT and middleware confusion	Built a small login demo first, then reused the same middleware for all routes
Wrong role access	Created a separate role middleware and tested each route with every role in Postman
CORS errors	Configured the cors library with the front-end URL
Database not connecting	Added the correct network access setting in MongoDB Atlas
Inconsistent responses	Used one standard JSON format and a central error handler

10.3 Current Limitations

- Document uploads are limited to text details; file upload storage is not yet included.
- Notifications appear inside the system only. SMS and push alerts are not supported yet.
- No online payment for fees.
- The free hosting plan may be slow after periods of no use.
- Automated test scripts are not yet written. Testing is mainly manual.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Upload and verification of documents such as ID proof and address proof.
- SMS, email and mobile push notifications.
- A mobile app for Android and iOS using the same API.
- Online payment of fees and bills.
- Support for multiple languages so that more citizens can use the platform.
- Admin dashboard with charts showing pending, solved and rejected cases.

11.2 Possible Integrations and Optimizations

- Integration with national identity and DigiLocker services for quick verification.
- Caching with Redis to make frequent requests faster.
- Splitting modules into separate microservices when the user base becomes very large.
- Automated testing and automated deployment pipelines.
- Priority and smart routing of complaints using simple rules or AI.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Citizen Service Management with Unified Backend Architecture project provides one digital platform for public services. Instead of separate systems for each task, a single backend manages citizens, service requests, complaints and notifications. The backend is exposed through secure REST APIs and protected with JWT authentication and role-based access.

12.2 What Was Achieved

- A working backend with all five planned modules.
- Secure registration, login and role-based access for three types of users.
- Online service requests and complaints with full status tracking.
- A notification system to share government announcements.
- A tested API collection and a live deployment on the cloud.
- Complete documentation that students can follow and reuse.

The expected outcome, a secure citizen service management platform that supports efficient public service delivery, has been met.

12.3 Skills Learned During Development

- Designing a modular backend and REST APIs with Node.js and Express.
- Working with MongoDB and Mongoose.
- Implementing authentication, authorization and password security.
- Testing APIs with Postman and fixing errors.
- Using Git and GitHub for teamwork.
- Deploying an application to the cloud and writing technical documentation.

12.4 Project Work Plan

Table 12.1: How the work was organised

Phase	Work done	Module
1	Study of the problem, requirements and tools	Planning
2	Database design and setup of the project folders	Design
3	Registration, login and profile features	Module 1
4	Service request creation and status update	Module 2
5	Complaints and notifications	Module 3
6	API security, validation and error handling	Module 4
7	Postman testing, deployment and report writing	Module 5

CHAPTER 13: REFERENCES

1. Node.js Documentation. https://nodejs.org/en/docs
2. Express.js Documentation. https://expressjs.com
3. MongoDB Manual. https://www.mongodb.com/docs
4. Mongoose Documentation. https://mongoosejs.com/docs
5. JSON Web Tokens Introduction. https://jwt.io/introduction
6. bcrypt package documentation. https://www.npmjs.com/package/bcrypt
7. Postman Learning Center. https://learning.postman.com
8. Render Documentation. https://render.com/docs
9. GitHub Docs. https://docs.github.com
10. MDN Web Docs: HTTP Methods and Status Codes. https://developer.mozilla.org
11. Digital India Programme. https://www.digitalindia.gov.in

CHAPTER 14: APPENDICES

A. Sample Code Snippets

Citizen model (models/Citizen.js)

```
const mongoose = require('mongoose');

const citizenSchema = new mongoose.Schema({
  name:     { type: String, required: true },
  email:    { type: String, required: true, unique: true },
  password: { type: String, required: true },
  phone:    String,
  address:  String,
  role:     { type: String, enum: ['citizen','officer','admin'],
              default: 'citizen' }
}, { timestamps: true });

module.exports = mongoose.model('Citizen', citizenSchema);
```

Authentication middleware (middleware/auth.js)

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.replace('Bearer ', '');
  if (!token) return res.status(401).json({ message: 'Login required' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
};
```

Role middleware (middleware/role.js)

```
module.exports = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ message: 'Not allowed' });
  next();
};
```

Complaint route (routes/complaintRoutes.js)

```
router.post('/', auth, role('citizen'), createComplaint);
router.get('/', auth, listComplaints);
router.put('/:id', auth, role('officer','admin'), updateComplaint);
```

B. Installation and Setup Instructions

1. Install Node.js and Git on your computer.
2. Download the project: git clone <repository-url>
3. Open the folder and run: npm install
4. Create a .env file with PORT, MONGO_URI, JWT_SECRET and JWT_EXPIRE.
5. Start the server with: npm start
6. Open Postman and send a request to http://localhost:5000/api/auth/login to confirm it runs.

C. User Manual

Citizen: register, log in, choose 'Apply for Service' or 'File Complaint', fill in the form and submit. Open 'My Requests' or 'My Complaints' to see the status. Open 'Notifications' to read announcements.

Officer: log in, open the list of assigned requests and complaints, review each one, update the status and write remarks.

Administrator: log in, manage users, assign complaints to officers, and publish notifications.

D. Simple Glossary

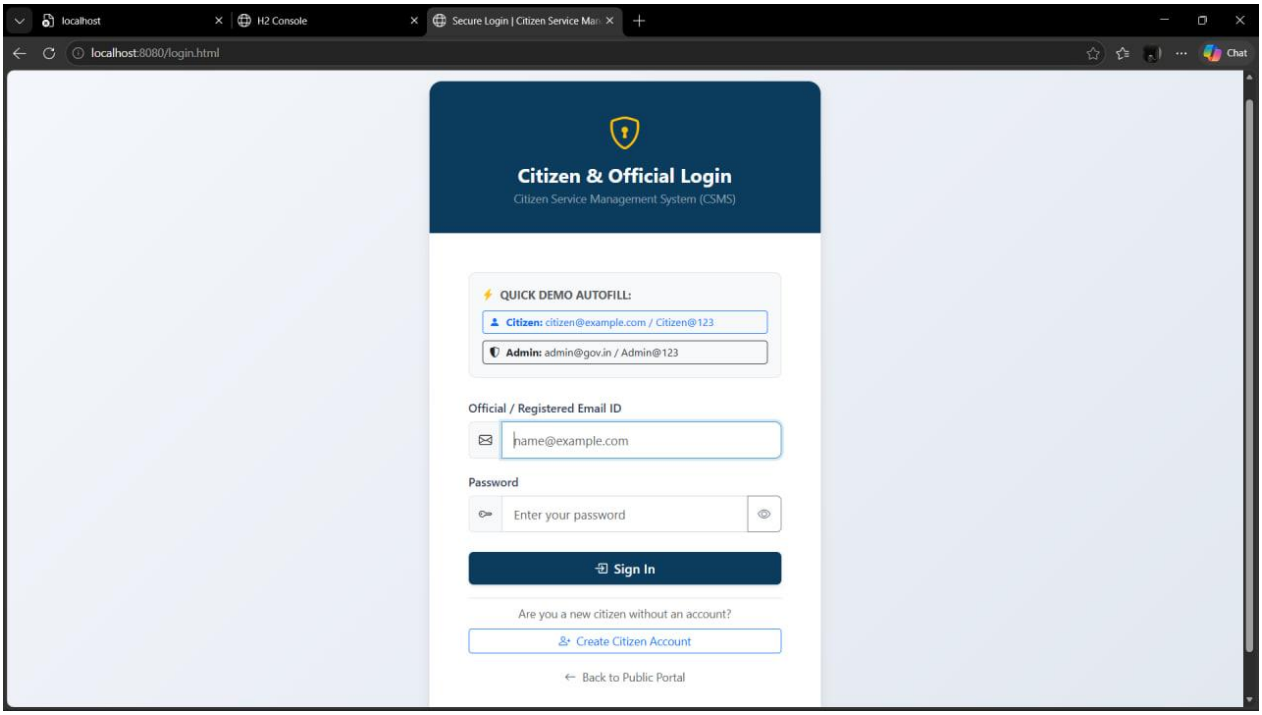
Table D.1: Common terms

Term	Meaning
Backend	Hidden part of the system that stores data and applies rules
Frontend	The screen and pages the user sees
API	A way for two programs to exchange data
REST	A common style of designing APIs using HTTP methods
JWT	A secure token proving that a user has logged in
Middleware	A function that runs before the main request handler
Hashing	Turning a password into unreadable text before storing it
Scalable	Able to handle more users without breaking
Deployment	Putting the project on the internet

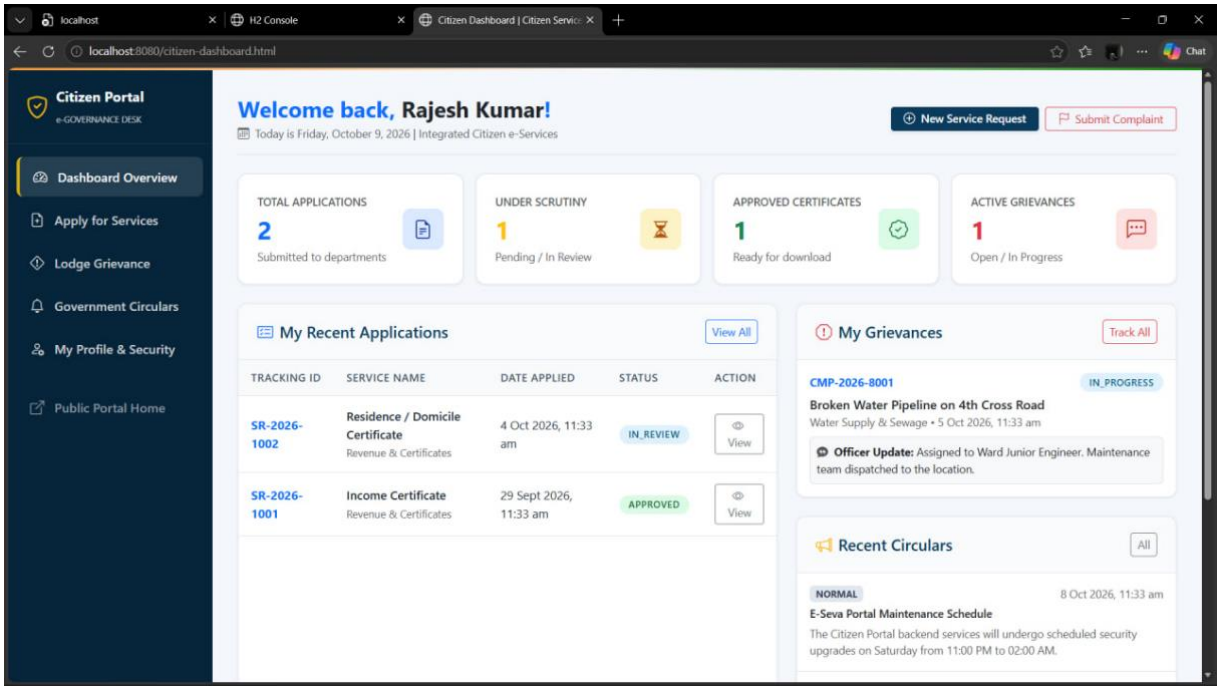
E. Screenshots

Replace each frame below with a screenshot of your running project and keep the caption.

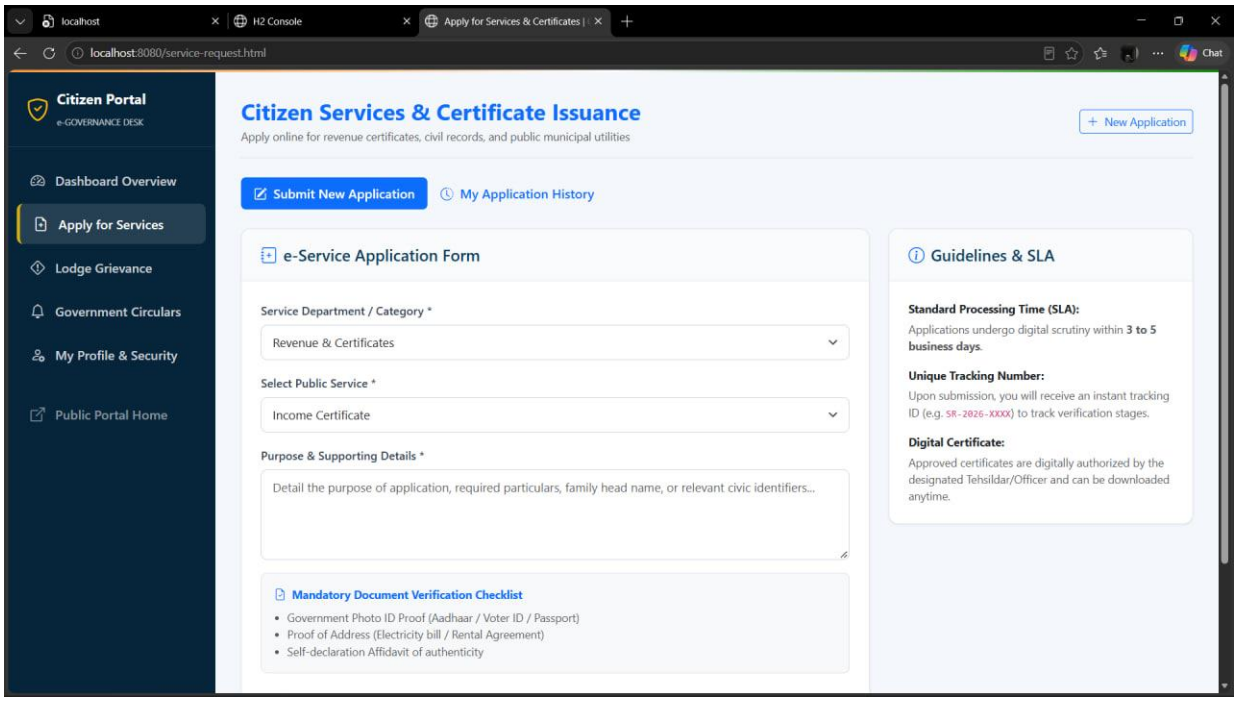
Screenshot 1: Registration and login page



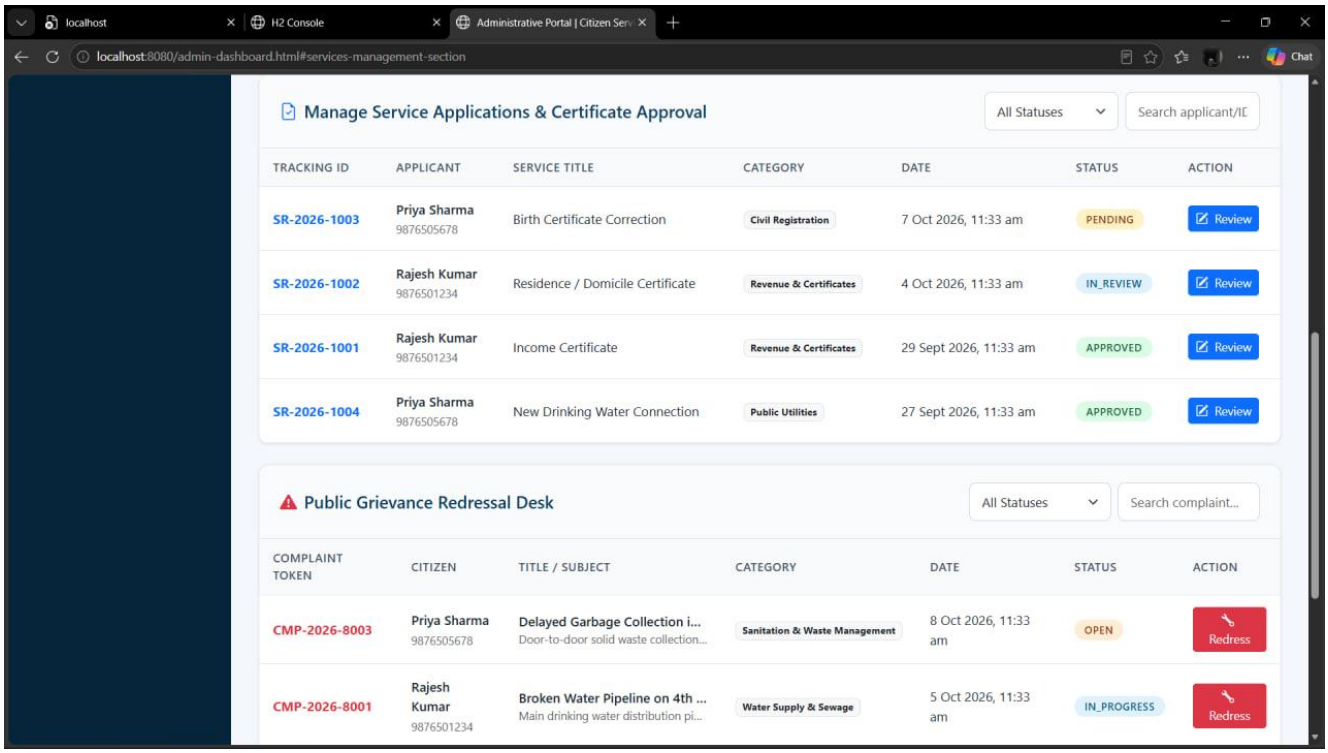
Screenshot 2: Citizen dashboard



Screenshot 3: Service request form



Screenshot 4: My Requests with status



Screenshot 5: Complaint form

